

PCA. As a third variant, we compute paired differences between AI-generated and human-written text activations and define the steering vector as the leading principal component of these differences:

$$v_l^{\text{pca}} = \text{PC}_1\left(\{a_l(x_j^{(f)}) - a_l(x_i^{(r)})\}\right).$$

This construction aims to identify the dominant axis associated with the human-to-AI-generated shift in representation space.

3.4 Projection features

Once steering vectors are obtained, we construct a feature representation of a text by using its alignment with these directions across layers. For each layer, we compute the layer-wise score as a cosine similarity between the pooled activation of this layer and its steering vector

$$s_l(x) = \frac{\langle a_l(x), v_l \rangle}{\|a_l(x)\|_2}.$$

The final feature vector is obtained by concatenating scores from all layers:

$$s(x) = (s_1(x), \dots, s_L(x)) \in \mathbb{R}^L.$$

Thus, instead of classifying directly from the full hidden states, we classify from a compact representation that summarizes how strongly the text aligns with AI-generated text directions throughout the model.

3.5 Detection head

To actually decide if the text is AI-generated, its projection features are fed into a lightweight downstream classifier. Our default detector consists of feature standardization followed by logistic regression. Given a feature vector $s(x)$, the detector outputs

$$p(y = 1 \mid x) = \sigma(w^\top \tilde{s}(x) + b),$$

where $\tilde{s}(x)$ is the standardized version of $s(x)$, $\sigma(\cdot)$ is the sigmoid function, and $y = 1$ denotes machine-generated text.

This choice isolates the effect of the steering-based representation from the complexity of the classifier. We ablate our choice of classifier considering alternative models in Sec. 4.3.

At test time, we extract activations for the new text, compute its projection features, and apply the trained classifier. Importantly, the reference LLM remains frozen throughout the entire pipeline.

4 Experiments

4.1 Experimental setup

Datasets. Our main evaluation uses three complementary benchmarks. Our primary benchmarks are **DetectRL** (Wu et al., 2024) and **MIRAGE** (Fu et al., 2025). DetectRL stresses robustness under distribution shifts across domains, source models, and attack families, while MIRAGE targets machine-writing scenarios, including generation, polishing, and rewriting. We then assess cross-benchmark generalization by transferring detectors between **DetectRL** and **MIRAGE**, and by evaluating detectors trained on these benchmarks zero-shot on **PADBen** (Zha et al., 2025). We additionally report results on the **COLING** dataset (Jawahar et al., 2020) in Appendix Sec. E, and cross-benchmark transfer from DetectRL and MIRAGE to the **RAID** dataset (Dugan et al., 2024) in Appendix Sec. F.4. We also use RAID for an additional matched-supervision comparison with RepreGuard (Chen et al., 2025), the closest representation-based detector in Sec. G.

DetectRL evaluation details. We evaluate on the three settings provided by DetectRL: (i) *Multi-Domain*, spanning ArXiv, XSum, Writing, and Review; (ii) *Multi-LLM*, spanning GPT-3.5, Claude, PaLM-2, and Llama-2; and (iii) *Multi-Attack*, spanning prompt-based, paraphrase, perturbation, and data-mixing attacks.

For Multi-Domain and Multi-LLM, AI-generated examples are balanced across latent factors by subsampling each (llm_type, data_type) combination, using 50 AI-generated samples per combination for Multi-Domain and 200 for Multi-LLM. For Multi-Attack, both human-written and AI-generated training sets are subsampled to 10,192 examples. We always evaluate on the full benchmark test split.

Within each setting, we report both **same-source** and **cross-source** evaluation: same-source trains and tests on the same domain, source model, or attack family, while cross-source trains on one source and evaluates on another.

MIRAGE evaluation details. We evaluate on MIRAGE under two settings, *DIG* (Disjoint-Input Generation) and *SIG* (Shared-Input Generation), and three transformations: GENERATE, POLISH, and REWRITE. The test splits are large-scale, e.g. 16,411 DIG-generate, 14,776 DIG-polish, 15,735 DIG-rewrite, 16,388 SIG-generate, 14,776 SIG-

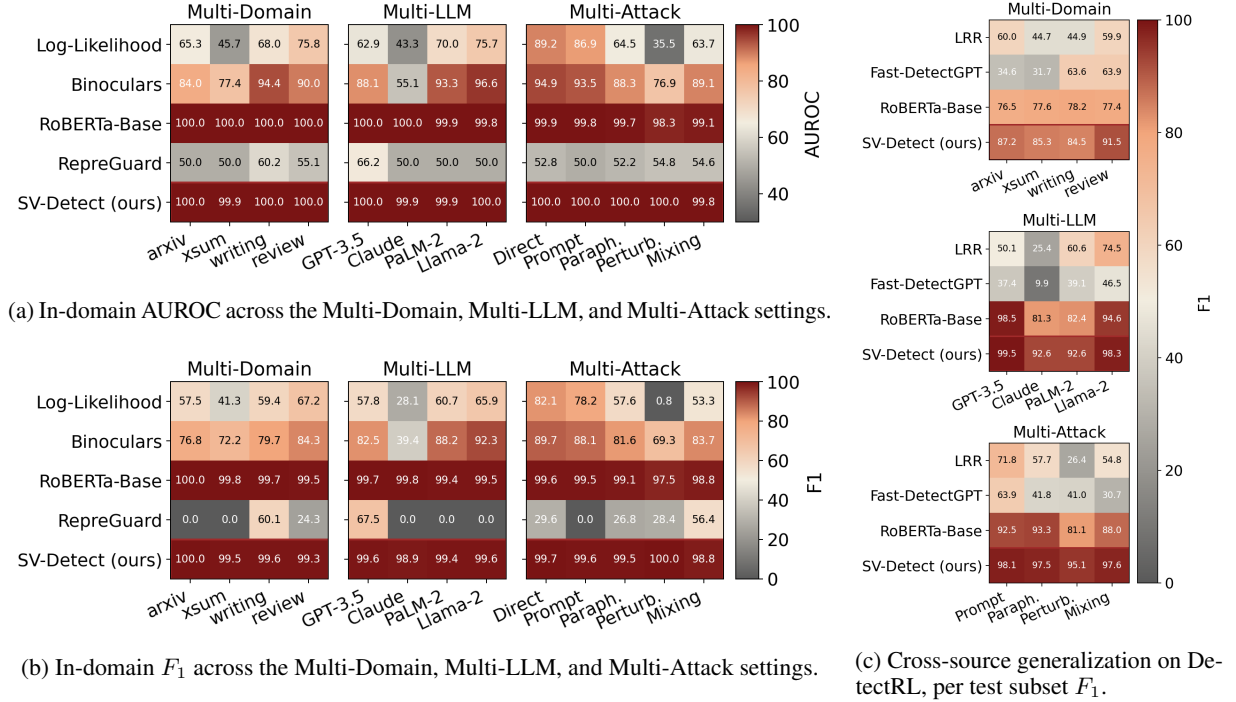


Figure 3: Performance on DetectRL.

polish, and 15,751 SIG-rewrite examples.

We use the original training setup of MIRAGE that contains 500 human-written/machine-generated pairs, where the machine-generated side is GPT-3.5-Turbo-polished text.

PADBen evaluation details. We use additional benchmark, PADBen (Zha et al., 2025), purely as zero-shot cross-benchmark target: our MIRAGE- and DetectRL-trained detectors are applied directly to PADBen test splits without any retraining or threshold recalibration.

PADBen targets paraphrase-based laundering and human-edited AI-generated text. We report sentence-pair AUROC on the five benchmark regimes (*paraphrase-source attribution*, *general authorship detection*, *AI-text laundering*, *iterative paraphrase depth*, *deep-paraphrase attack*).

Evaluation protocol. Following recent work (Fu et al., 2025; Chen et al., 2024; Bao et al., 2023; Mitchell et al., 2023), for fair comparison with baseline methods, in all our main experiments, the reference model used for activation extraction is a frozen GPT-Neo-2. 7B (Black et al., 2021). For the RAID matched-supervision comparison against RepreGuard we additionally report results with a frozen LLaMA-3.1-8B backbone to match RepreGuard’s original setup. We ablate choice of backbone model in Appendix C. Texts are tokenized with truncation to a maximum length of 2048 to-

kens. For each text, we extract the mean-pooled hidden representation from every layer and compute cosine-similarity features with the corresponding steering vectors, as described in Section 3. We use mean pooling because it is simple, stable, and computationally cheap; richer summary statistics such as max pooling, attention-weighted pooling, or sentence-level aggregation are promising extensions but are left to future work. The downstream detector is trained on the same train data split used to construct the steering vectors.

Unless stated otherwise, the final detector is a pipeline consisting of StandardScaler followed by LogisticRegression with liblinear solver, ℓ_2 -regularization parameter $C = 1.0$, and random seed 42. For evaluation, we report AUROC, AUPR, TPR@FPR=5%, Balanced Accuracy, Matthews correlation coefficient (MCC), and F_1 . Full numerical tables corresponding to all result figures in this section are provided in Appendix Sec. F.

4.2 Experimental results

DetectRL in-distribution performance. We follow (Wu et al., 2024) and compare SV-Detect against both zero-shot and supervised baselines. In the same-source setting, the zero-shot baselines are Log-Likelihood (Solaiman et al., 2019), Entropy (Lavergne et al., 2008), Rank, Log-Rank (Gehrmann et al., 2019), LRR, NPR (Su

et al., 2023), DetectGPT (Mitchell et al., 2023), DNA-GPT (Yang et al., 2023), Revise-Detect (Zhu et al., 2023), Binoculars (Hans et al., 2024), and Fast-DetectGPT (Bao et al., 2023), while the supervised baselines are RoBERTa-Base (Park et al., 2021), RoBERTa-Large, XLM-RoBERTa-Base, and XLM-RoBERTa-Large (Conneau et al., 2019). We additionally add RepreGuard (Chen et al., 2025) to the comparison, which is the closest supervised baseline to our work. For the fair comparison, we implemented RepreGuard using matched GPT-Neo-2.7B backbone based on the official provided code. Comparison of SV-Detect with RepreGuard using the default backbone of RepreGuard (LLaMA-3.1-8B-Instruct) is provided in Appendix Sec. G.

For readability, Fig. 3a and Fig. 3b show a representative subset of baselines alongside SV-Detect: Log-Likelihood as a simple zero-shot scoring baseline, Binoculars as the strongest zero-shot detector, RoBERTa-Base as a strong supervised encoder baseline, and RepreGuard as the closest representation-based method. The full per-subset breakdown over all baselines is reported in Appendix Sec. F.

SV-Detect achieves near-perfect performance across all three DetectRL settings, with AUROC ranging between 99.8 and 100.0 and F1 between 98.8 and 100.0.

These results show that SV-Detect matches or slightly exceeds the strongest supervised baselines in the same-source setting, where performance is already close to saturated. At the same time, it substantially outperforms zero-shot and representation-based baselines, including Binoculars and RepreGuard, whose performance varies more strongly across domains, source models, and attack families.

DetectRL cross-setting generalization. We next evaluate cross-source generalization following (Wu et al., 2024). In this setting, a detector is trained on one source condition and evaluated on a different source condition within the same DetectRL setting. We compare against the baselines reported in (Wu et al., 2024): LRR and Fast-DetectGPT as zero-shot detectors, and RoBERTa-Base as a supervised baseline.

Fig. 3c reports mean F_1 for each detector and evaluation subset, averaging over all train-test pairs in which the training subset differs from the evaluation subset. This isolates robustness to source mismatch: each column asks how well a detector performs on a target subset when it has not been

trained on that subset.

SV-Detect achieves the strongest and most consistent cross-source performance across all three DetectRL settings. On *Multi-Domain*, it improves over RoBERTa-Base by roughly 7–14 F_1 points across target domains. On *Multi-LLM*, it remains above 92 F_1 for every target model and reaches near-perfect transfer on GPT-3.5 and Llama-2. On *Multi-Attack*, it is similarly robust, maintaining at least 95 F_1 across all attack families. In contrast, the zero-shot baselines vary substantially across target subsets, and RoBERTa-Base degrades more noticeably under source mismatch.

Overall, these cross-source results support the main claim of our approach: AI-generated text directions extracted from hidden representations encode a transferable human-vs-AI signal, enabling strong generalization across domains, source models, and attack families. Full train-test matrices are reported in Appendix Sec. F.

MIRAGE results. On MIRAGE, following (Fu et al., 2025), we compare SV-Detect against Log-Likelihood, LogRank, Entropy, RoBERTa-Base, RoBERTa-Large, LRR, DNA-GPT, NPR, DetectGPT, Fast-DetectGPT, ImBD (Chen et al., 2024), and DetectAnyLLM (Fu et al., 2025). As in the DetectRL experiments, we additionally include RepreGuard (Chen et al., 2025) with a matched GPT-Neo-2.7B backbone as the closest representation-based baseline.

As seen on Fig. 4, SV-Detect performs strongly across all six MIRAGE settings and achieves the best results on direct generation, with AUROC 0.9777 on DIG-GENERATE and 0.9779 on SIG-GENERATE. On POLISH and REWRITE, it remains competitive but is outperformed by DetectAnyLLM.

In Appendix Sec. F.4, we report an additional experiment on MIRAGE, where we use additional training data to construct separate task-specific per-layer steering directions corresponding to GENERATE, POLISH, and REWRITE subtasks. This variant substantially improves performance on the editing subtasks, showing that task-specific directions provide a richer representation of the different MIRAGE transformation types than the default POLISH-only direction.

Cross-benchmark transfer. We further analyze whether the representations learned by SV-Detect transfer across benchmark boundaries. Specifically, we train SV-Detect on one benchmark and eval-